

思考题与习题七

6.2 试说明并行接口和串行接口在数据传输和内部结构上的主要区别。

解：并行接口使传送数据的各位同时在总线上传输，而串行接口则使数据一位一位地传输。两种接口在结构和功能上的主要差别在于：串行接口需进行并行与串行之间的相互变换，而并行接口无需进行这种变换。

7.28 8255 的工作方式控制字和按位置位/复位控制字都是写入控制端口的，它们是如何区分的？

解：由写入控制端口的控制字的最高位 D_7 来区分，若控制字最高位 D_7 为 1，说明写入的是工作方式控制字，否则写入的是按位置位/复位控制字。

7.29 8255 的方式 0 一般使用在什么场合？在方式 0 时，能否使用应答式方式传送数据？若能，如何实现？

解：方式 0 一般使用在不需要应答联络的无条件传送，但也可使用应答方式传送数据。采用应答式传送时，一般用 A 口或 B 口作为数据口，没有固定的应答线，而是由程序设定 C 口的高低 4 位分别作为应答的控制和状态信息通道。

7.31 设 8255 的端口 A、B、C 和控制寄存器的地址为 $f4h$ 、 $f5h$ 、 $f6h$ 、 $f7h$ ，要使 A 口工作于方式 0 输出，B 口工作于方式 1 输入，C 口上半部输出，下半部输入，且要求初始化时使 $PC_6=0$ ，试设计 8255 与 PC 系列机的接口电路，并编写初始化程序。

解：假定用门电路译码，8255 与 PC 系列机的接口电路可如图 7.26 所示。相应的汇编语言初始化程序如下：

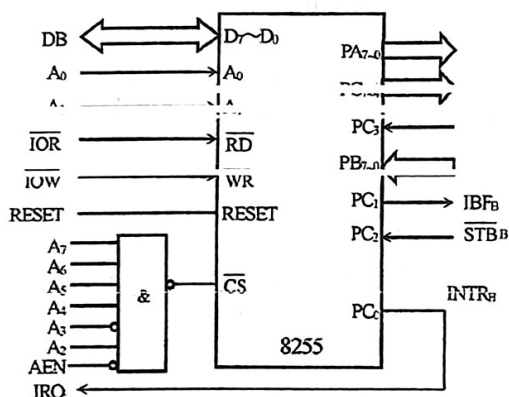


图 7.26

```
mov al, 10000111b      ; 方式字
out 0f7h, al
mov al, 00001100b      ; PC6=0
out 0f7h, al
mov al, 00000101b      ; 开中断
out 0f7h, al
```

若用 C 语言编程，则程序如下：

```
outportb(0xf7, 0x87); /* 写 8255 方式控制字 */
outportb(0xf7, 0x0c); /* PC6=0 */
outportb(0xf7, 0x05); /* 开 INTRB 中断 */
```

7.35 设系统机外扩一片 8255 以及相应的实验电路，如图 7.27 所示。要求：先预置开关 $K_3 \sim K_1$ 为一组状态，然后按下自复按钮 K 产生一个负脉冲信号，输入到 PC_4 。用发光二极管 LED_i 亮，显示 $K_3 \sim K_1$ 的状态。重复以上操作，直到主机键盘有任意键按下时结束演示。

要求： $K_3 K_2 K_1 = 000$ 时， LED_1 亮， $K_3 K_2 K_1 = 001$ 时， LED_2 亮

$K_3K_2K_1=010$ 时, LED₃ 亮, $K_3K_2K_1=011$ 时, LED₄ 亮
 $K_3K_2K_1=100$ 时, LED₅ 亮, $K_3K_2K_1=101$ 时, LED₆ 亮
 $K_3K_2K_1=110$ 时, LED₇ 亮, $K_3K_2K_1=111$ 时, LED₈ 亮

$K_3 \sim K_1$ 闭合为 0, 断开为 1。

- (1) 试分析该接口电路中 A 端口、B 端口应工作在什么方式下?
- (2) 试编写完成上述功能的程序。

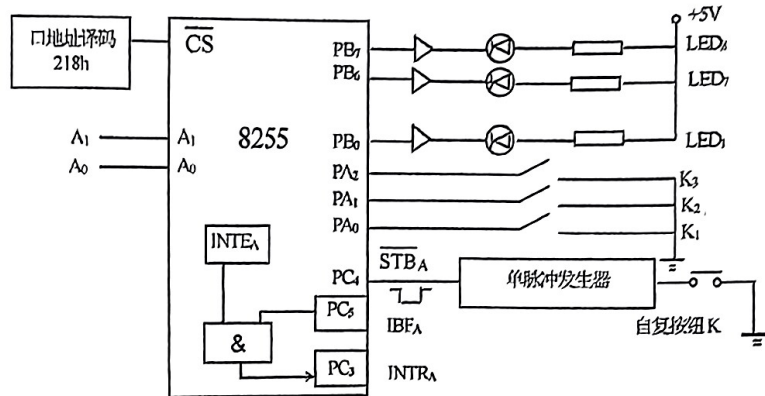


图 7.27 8255 实验电路

解: (1) 图中 8255 的 B 口用于控制发光二极管显示, 应工作在方式 0 输出; 而 A 口采用固定握手联络信号的选通输入方式来输入开关状态, 但并未使用中断, 所以 A 口应工作在方式 1 查询输入。

(2) 图中 8255 的端口地址由 A_0 、 A_1 选择, 片选基地址为 218h, 由此可以确定 8255 的地址为: 218h~21bh。关键方法是在内存中建立一个显示控制字表, 当读入开关状态后, 通过查表获取与开关状态相对应的显示控制字。相应的汇编语言工作程序如下:

```
data segment
    msg db '8255 READY.....', 0dh, 0ah, '$'
    tab db 11111110b, 11111101b, 11111011b, 11110111b
        db 11101111b, 11011111b, 10111111b, 01111111b
data ends

code segment
    assume cs:code, ds:data
start:  mov ax, data
        mov ds, ax
        mov dx, 21bh
        mov al, 10110000b          ; A 口方式 1 输入、B 口方式 0 输出
        out dx, al
        mov al, 08h               ; PC4 置 0, 使 INTEA=0, 关中断
        out dx, al
        mov ah, 9
        mov dx, offset msg
        int 21h                   ; 给出操作提示
scan:   mov ah, 1
        int 16h
        jnz return                ; 有键按下, 退出
        mov dx, 21ah              ; 取 C 口地址
        in al, dx                  ; 读 A 口方式 1 状态字
        test al, 20h              ; 有数据输入 (IBFA=1) ?
```

```

        jz     scan                ; 无, 继续查询等待
        mov    dx, 218h            ; 读开关状态 (A 口)
        in     al, dx
        mov    bx, offset tab      ; 取显示控制码表首址
        and    al, 07h             ; 保留有效开关状态 (K2K1K0)
        xlat                     ; 查表, 将开关状态转换成显示控制码
        inc    dx
        out    dx, al              ; 显示开关状态
        jmp    scan
return:  mov    ah, 4ch
        int    21h
code    ends
        end    start

```

若用 C 语言编程, 则程序如下:

```

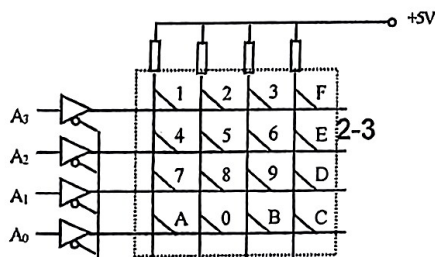
#include <stdio.h>
#include <dos.h>
unsigned char tab[]={0xfe,0xfd,0xfb,0xf7,0xef,0xdf,0xbf,0x7f};
main(){
    unsigned char i,status;
    outportb(0x21b,0xb0);          /* 写 8255 方式控制字 */
    outportb(0x21c,0x00);          /* 初始化 A 口为输入 */
    printf("8255 READY.....\n"); /* 显示开关状态 */
    while(!kbhit()){               /* 无键按下时, 循环 */
        status=inportb(0x21a);      /* 从 C 口输入状态 */
        if(status&0x20){            /* 当 IBFA=1 时, 读开关状态 */
            i=inportb(0x218);       /* 从 A 口输入开关状态 */
            outportb(0x219,tab[i&0x7]);
        }
    }
}

```

8.6 IBM—PC 总线上有一键盘输入接口如图 8.22 所示, 请问:

- (1) 该 I/O 口的基地址是唯一的吗? 试列出一个有效的基地址;
- (2) 要判断是否有键按下, 扫描地址应是什么?
- (3) 如按下‘9’键, 扫描地址是什么? 从该地址读入的值又是什么?

解: 该接口利用低端线地址线 $A_3 \sim A_0$ 用作行扫描输出信号, 三态门接口用作列状态输入, 按键识别的基本原理是根据扫描信号 $A_3 \sim A_0$ 确定的扫描地址, 读入列状态来判断按键的行/列位置、识别按键。由此可解:



(1) 图中使用“与非门”产生三态门接口地址译码信号，当 $A_9A_8A_7A_6A_5A_4=010000$ 时，端口被选中。由于 $A_{15} \sim A_{10}$ 未参入高端地址译码，所以在 IBM-PC 总线 I/O 空间，该接口的基地址不唯一，若取 $A_{15} \sim A_{10}=000000b$ ，则基地址为 100h，这时地址范围为：100h~10fh。

(2) 图中 $A_3 \sim A_0$ 用作行扫描输出，若要判断是否有键按下，应使行扫描输出信号 $A_3 \sim A_0=0000$ ，相应的扫描地址应为 100h；

(3) 4 个有效的行扫描地址应为：10eh、10dh、10bh 和 107h。这时要产生含‘9’键这一行的行扫描信号，应使 $A_3 \sim A_0=1101$ ，对应的扫描地址是 10dh；若从该接口读入列状态，则‘9’所在列 D_2 的状态应为 0，此时读入的值应为 $\times \times \times \times 1011b$ 。

8.9 有如图 8.19 所示 8 位 8 段 LED 显示器。试为该显示器设计一个按动态扫描、分时显示原理工作的 LED 显示器接口，编写程序显示“1927.8.1”8 个字符。



图 8.19 8 位 LED 显示器接口

解：要实现按动态扫描、分时显示原理工作的 8 位 LED 显示器接口，需 2 个 8 位并行输出口，若选用 8255 实现，则相应的接口电路可如图 8.20 所示。8255 各端口地址为 200h~203h，A 口用于段码输出，B 口用作位码输出端口。LED 采用共阴极接法，但各阳极字段与 8255 的 A 口之间增加了一级反向驱动器，所以字符显示的段码仍应采用共阳极段码。

基于上述动态扫描循环显示思想的汇编语言显示驱动程序如下：

```
data segment
    segpt      db 0c0h, 0f9h, 0a4h, 0b0h      ; 定义共阳极显示段码表
               db 99h, 92h, 82h, 0f8h
               db 80h, 90h, 88h, 83h
               db 0c6h, 0a1h, 86h, 8eh, 7fh
    dismem db 1, 9, 2, 7, 10h, 8, 10h, 1 ; 显示缓冲区 (1927.8.1)
data ends
```

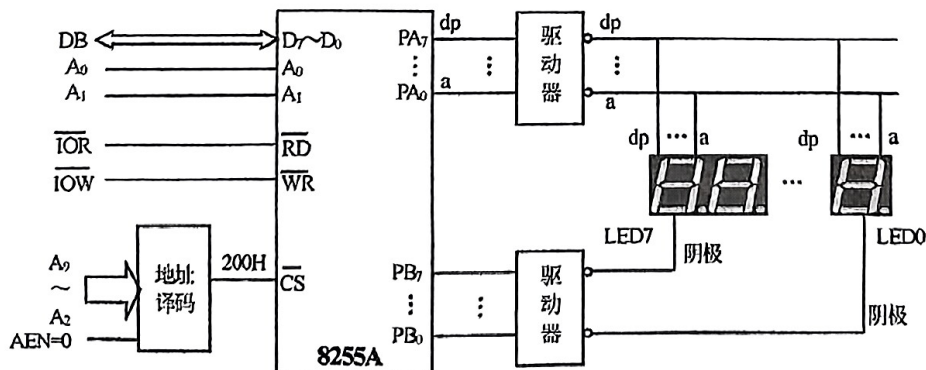


图 8.20 8 位 LED 显示器接口

```

code segment
    assume cs: code, ds: data
start:    mov     ax, data
          mov     ds, ax
          mov     dx, 203h                ; 控制寄存器地址
          mov     al, 10000000b           ; A 口和 B 口均为方式 0 输出
          out     dx, al
again:    mov     di, offset dismem        ; 指向显示缓冲区首址
          mov     cl, 80h                 ; 指向左端 LED 显示器
          mov     al, 00h                 ; 将 00 送位码寄存器, 关显示
          mov     dx, 201h
          out     dx, al
disp:     mov     al, [di]                ; 取要显示的字符
          mov     bx, offset segpt        ; 段码表首址送 bx
          xlat                                     ; 查表获取显示字符的显示段码
          dec     dx
          out     dx, al                  ; 将段码送至端口 A
          mov     al, cl                  ; 取当前位码
          inc     dx
          out     dx, al                  ; 将位码送端口 B
          mov     cx, 0ffffh           ; 延时, 以便得到稳定的显示效果
          mov     cx, 0ffffh
delay:    nop
          loop    delay
          pop     cx                      ; 从堆栈取出位码
          cmp     cl, 01                  ; 显示至最右端了吗
          jz      disend                  ; 是, 转出口
          inc     di                      ; 否, 指向下一位要显示的字符
          shr     cl, 1                   ; 位码右移一位, 指向下一个数位
          jmp     disp
disend:   mov     ah, 1                   ; 检测 PC 键盘是否有键按下
          int     16h
          jz      again                  ; 无键按下, 继续新一轮显示
          mov     ah, 4ch                 ; 返回 DOS
          int     21h
code      ends
end       start

```

若用 C 语言编程, 则完成上述显示功能的驱动程序如下:

```

#include "stdio.h"
#include "conio.h"
#include "dos.h"
#define port 0x200 /* 定义 8255 基地址 */
unsigned char segpt[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,
                      0x80,0x90,0x88,0x83,0xc6,0xa1,0x86,0x8e,0x7f};

main(){

```

```

unsigned char dismem[]={1,9,2,7,0x10,8,0x10,1};
unsigned char temp,data,i;
int count;
outportb(port+3,0x80);          /* 初始化 8255 */
while(!kbhit()){                /* 无键按下, 循环 */
    i=0;
    data=0x80;                  /* 位码指向左端 LED 显示器 */
    outportb(port+1,0);         /* 关显示 */
    while(1){
        temp=dismem[i];         /* 取要显示的字符 */
        outportb(port,segpt[temp]); /* 将显示字符的显示段码送端口 A */
        outportb(port+1,data);  /* 将位码送端口 B */
        count=0xffff;
        while(count!=0) count--; /* 延时 */
        if(data==0x01) break;
        i++;                    /* 指向下一位要显示的字符 */
        data>>=1;               /* 位码右移一位 */
    }
}
}

```

8.10 有如图 8.23 所示的键盘电路, 试编写 8255 初始化程序和键值读取程序, 并将键值序号在 LED 七段数码管显示出来。

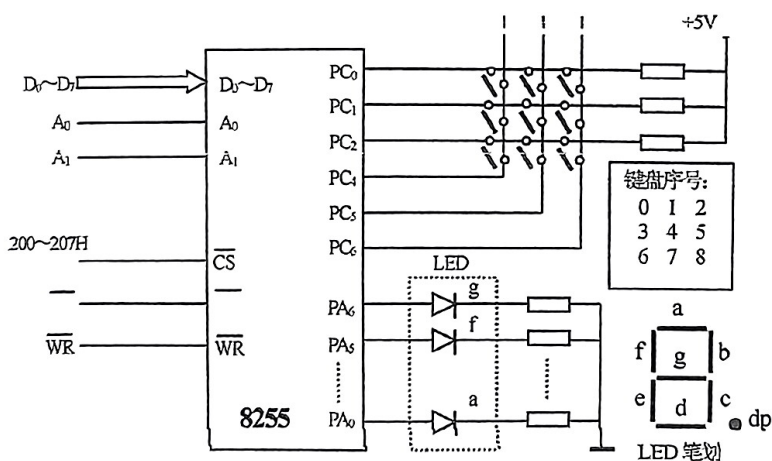


图 8.23 键盘和 LED 显示器接口

解: 图中采用列扫描键盘, C 口的 $PC_6 \sim PC_4$ 用于输出列扫描信号, $PC_2 \sim PC_0$ 用于输入行状态信号, 所以 C 口的方式为高 4 位输出、低 4 位输入。A 口驱动数码管显示为方式 0 输出。

要判断是否有键按下的方法是通过 $PC_6 \sim PC_4$ 输出三个为 0 的列扫描信号, 然后从 $PC_2 \sim PC_0$ 读入行状态, 若有键按下则对应的状态必然为 0; 若无键按下则所有行状态均为高电平 1。检测到有键按下时需延时去抖动, 然后依次扫描 3 列, 对应列扫描信号为: 11101111B、11011111B、10111111B。假定从上到下行号分别为 0~2, 从左到右列号分别为 0~2, 则识别到按键时, 将对应行号乘以 3 加上列号即为键盘序号。如是可编写用列扫描识别按键并通过数码管显示的程序如下:

```

data segment
    segtab db 3fh, 06h, 5bh, 4fh          ; 定义 0~8 的共阴极显示段码
            db 66h, 6dh, 7dh, 07h, 7fh

```

```

data ends
code segment
    assume cs:code, ds:data
start:    mov     ax, data
          mov     ds, ax
          mov     dx, 203h           ; 取 8255 控制寄存器地址 (或 207h)
          mov     al, 10000001b      ; 8255A 方式控制字
          out     dx, al
          mov     al, 0              ; 初始数码管不显示 (A 输出全 0)
          mov     dx, 200h
          out     dx, AL
key:      mov     dx, 202h           ; 取 C 口地址
          mov     al, 0
          out     dx, al             ; 输出所有列为 0 的扫描信号
          in      al, dx             ; 读行状态 (PC2~PC0)
          not     al                 ; 状态取反, 为 1 表示有键按下
          test    al, 07h           ; 有键按下? (PC2~PC0=000?)
          jz      key               ; 无, 继续扫描
          call    delay             ; 软件延时 20ms, 去抖动
          in      al, dx             ; 读列状态
          not     al
          test    al, 07h           ; 有键按下?
          jz      key               ; 无, 表示无按键信号, 继续扫描
          mov     ch, 0efh          ; 置列扫描信号, 从第 0 列开始
          mov     bl, 0              ; 记录列号
again:    mov     al, ch
          out     dx, al             ; 输出列扫描信号
          in      al, dx             ; 读当前列按键状态
          not     al                 ; 行状态取反, 1 表示有键按下
          and     al, 07h           ; 当前列有键按下?
          jnz     next              ; 有, 转 next
          inc     bl                 ; 列号加 1
          rol     ch, 1              ; 形成下一列扫描信号
          cmp     ch, 7fh           ; 扫描完 3 列?
          jz      key               ; 扫描完转 key, 继续等待
          jmp     again             ; 继续列扫描
next:     mov     bh, 0              ; 计算行号, 初值为 0
keyma:    test    al, 01h           ; 当前行有键按下?
          jnz     exit              ; 有键按下, bh 为行号
          shr     al, 1              ; 指向下一行
          inc     bh                 ; 行号加 1
          jmp     keyma
exit:     mov     al, bh             ; 计算键号: (bh)×3+(bl)→al
          shl     al, 1
          add     al, bh
          add     al, bl
          mov     bx, offset segtab; 取段码表首址

```

```

        xlat                                ; 键号转换成相应显示段码
        mov     dx, 200h                    ; 从 A 口输出显示
        out     dx, al
        mov     ah, 1                        ; 检测 PC 键盘是否有键按下
        int     16h
        jz      key                          ; 无键按下, 继续新一轮扫描
        mov     ah, 4ch
        int     25h
delay:   push    cx                          ; 延时子程序
        mov     cx, 0
delay1:  loop    delay1
        pop     cx
        ret
code     ends
        end     start

```

若用 C 语言编程, 则完成上述按键识别和显示功能的驱动程序如下:

```

#include "dos.h"
delay20ms(){                                /* 延时子程序 */
    int count;
    count=0xffff;
    while(count!=0) count--;
}

main(){
    /* 定义 0~8 的共阴极显示段码 */
    unsigned char segtab[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f};
    unsigned char status,datach,row,low,i;
    int count;
    outportb(0x203,0x81);                    /* 初始化 8255 */
    while(!kbhit()){                          /* 无键按下, 循环 */
        outportb(0x202,0);                    /* 输出 PC6~PC4 为 0 的扫描信号 */
        status=inportb(0x202);                /* 读行状态 (PC2~PC0) */
        status^=0xff;                          /* 状态取反 */
        if(!(status&0x07)) continue;          /* 无键按下, 继续新一轮扫描 */
        delay20ms();                          /* 软件延时去抖动 */
        detach=0x10;
        low=0;
        while(low<3){
            outportb(0x202,~detach);           /* 输出当前列扫描信号 */
            status=inportb(0x202);             /* 读行状态 (PC2~PC0) */
            status^=0xff;                      /* 状态取反 */
            status&=0x07;
            if(status!=0) break;               /* 有键按下, 退出扫描 */
            low++;
            detach<<=1;
        }
        if(low==3) continue;
    }
}

```



```

row=0;
while(!(status&0x01)){
    status>>=1;
    row++;
}
i=row*3+low;          /* 计算按键序号 */
outportb(0x200, segtab[i]); /* 显示按键 */
}
}

```

7.39 INS8250 中有多少个可访问的寄存器和多少个端口地址? 请写出它们的对应关系。从已学过的各种可编程接口芯片中, 试总结出一般可编程接口芯片中是如何解决寄存器多、端口地址少的矛盾的。

解: INS8250 中有 10 个 8 位寄存器, 但只提供了 8 个端口地址, 实际只用到 7 个地址。各端口与寄存器对应关系如表 7.4 所示。

表 7.4 INS 8250 内部可读/写寄存器及其访问控制

DLAB 位	A ₂	A ₁	A ₀	读/写	访问的寄存器
0	0	0	0	读	接收缓冲器
0	0	0	0	写	发送保持器
0	0	0	1	读/写	中断允许寄存器
×	0	1	0	读	中断标识寄存器
×	0	1	1	读/写	线路控制寄存器
×	1	0	0	读/写	MODEM 控制寄存器
×	1	0	1	读/写	除数寄存器(低字节)
×	1	1	0	读/写	MODEM 状态寄存器
1	0	0	0	与	除数寄存器(高字节)
1	0	0	1	写	除数寄存器(高字节)

各种可编程接口芯片中, 解决内部读/写寄存器多、端口地址线少的矛盾的常用方法有:

① 利用命令字中的某一位(或某些位)的状态作为标志, 在同一地址下, 通过不同的状态寻址不同的内部寄存器。

② 在同一端口地址下, 利用读写的顺序来决定写入或读出不同的内部寄存器。

③ 一个只读寄存器和一个只写寄存器可以共用同一个地址, 然后利用读命令(产生读信号)和写命令(产生写信号)来区分访问的是只读寄存器还是只写寄存器。

④ 用一个专门的指示器, 在同一端口地址下, 按指示器的不同状态, 来访问不同的内部寄存器。

⑤ 利用某个命令字事先加以指定, 而后寻址同一地址可访问不同的寄存器。

7.46 采用 8250 实现甲乙两站之间的近距离通信, 甲方发送, 乙方接收, 传送的数据块为 2KB。要求:

(1) 设计通信接口电路, 画出系统硬件连接图;

(2) 完成对 8250 的初始化编程;

(3) 编写甲方的发送程序和乙方的接收程序。

解: (1) 甲、乙双方均可采用如图 7.27 所示接口。通信时可通过 RS-232 接口实现甲乙两站之间的连接。

(2) 假定通信波特率为 4800 (除数寄存器值 = $\frac{f_{\text{基准时钟}}}{\text{波特率} \times 16} = \frac{1.8423 \times 10^6}{4800 \times 16} = 0 \times 18$), 每个字符由 1 位起

始位、7 位数据位, 1 个偶校验位、2 个停止位组成, 则相应的 8250 初始化程序如下:

```

init8250(int base){          /* 8250 基地址 */
    outportb(base+3, 0x80); /* 1cr=1, 允许访问除数寄存器 */
}

```

```

outportb(base,0x18);          /* 写除数寄存器低位 */
outportb(base+1,0);          /* 写除数寄存器高位 */
outportb(base+3,0x1e);       /* 7 个数据位、偶校验、2 个停止位 */
outportb(base+4,0x03);       /* 设置数据终端就绪、请求发送 */
outportb(base+1,0);          /* 屏蔽中断 */
}

```

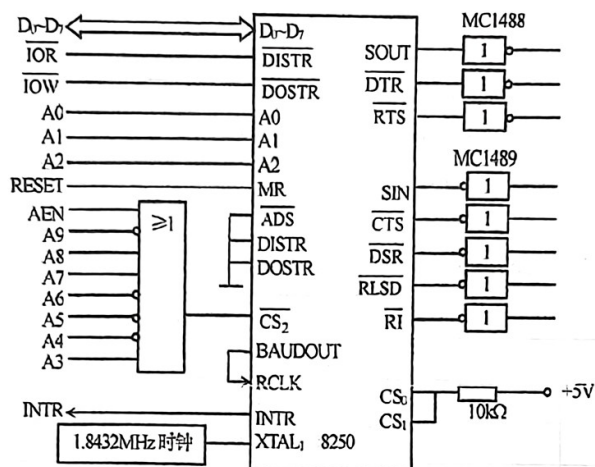


图 7.27 8250 与系统总线的连接

(3) 假定采用查询通信，则相应的甲方发送程序和乙方接受程序如下：

甲方发送程序：

```

#include "conio.h"
#include "dos.h"
#define base 0x278
char sbuf[2048];          /* 发送缓冲区 2KB */
main()
{
    unsigned char status,i;
    init8250(base);       /* 8250 初始化 */
    i=0;
    while(i<2048){
        status=inportb(base+5); /* 读线路状态寄存器 lsr */
        if(status&0x1e)
            printf("error\n"); /* 出错，显示错误提示信息 */
        else
            if(status&0x20){
                /* 发送保持器空 */
                outportb(base,sbuf[i]); /* 从发送缓冲区取 1 字符并发送 */
                i++;
            }
    }
}

```

乙方接收程序：

```

#include "stdio.h"
#include "conio.h"
#include "dos.h"
#define base 0x278

```

```

char rbuf[2048];                                /* 接收缓冲区 2KB */
main(){
    unsigned char status,i;
    init8250(base);                             /* 8250 初始化*/
    i=0;
    while(i<2048){
        status=inportb(base+5);                 /* 读线路状态寄存器 lsr */
        if(status&0x1e)
            printf("error\n");                 /* 出错，显示错误提示信息 */
        else
            if(status&0x01){                     /* 接收缓冲器满 */
                rbuf[i]=inportb(base);           /* 读接收缓冲器并送接收缓冲区 */
                i++;
            }
    }
}

```